Anexo Evaluación 3

Unidad 3: API restful con integración a base de datos.

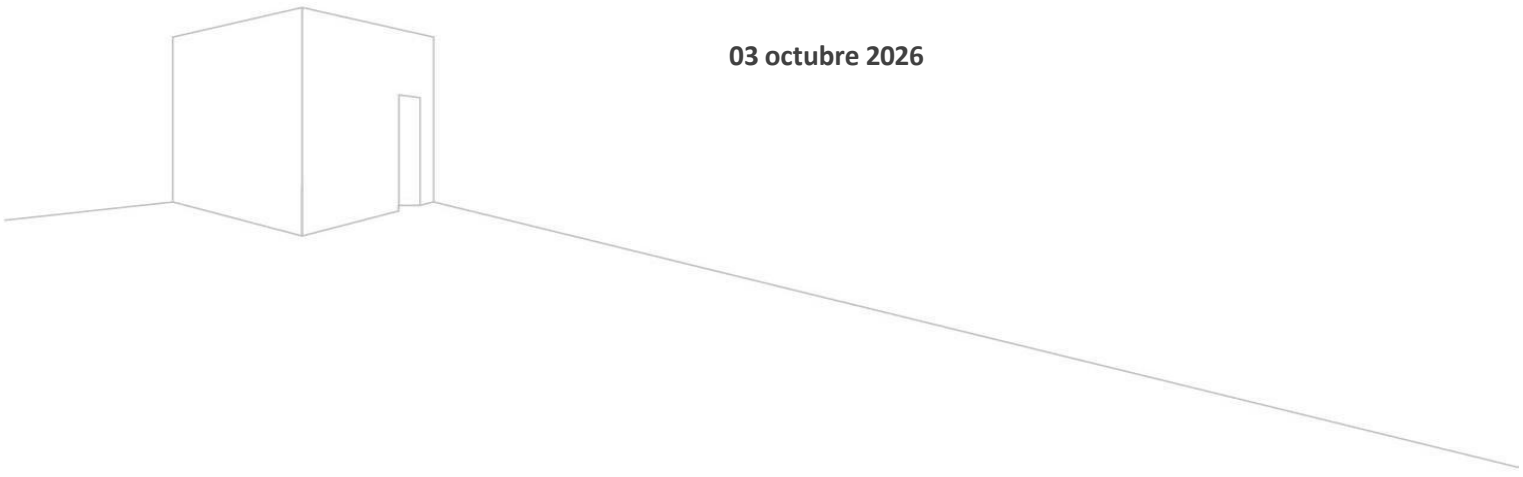
Asignatura: Programación Back End

Sección: 2026/P TI3V41/V-IEI-N4-P3-C1/V La Granja IEI

Nombre del docente: Fabio Vivas

Nombre de los integrantes del grupo: Constanza Cortés

03 octubre 2026



Contenido

Capítulo 1 Anexo 3

Capítulo 2 Referencias bibliográficas 5

Capítulo 1
Anexo

1.0 Anexo: Declaración de Uso de Inteligencia Artificial

Para el desarrollo del presente proyecto evaluativo, y en estricto apego a las instrucciones de la Evaluación 3 que permiten el uso controlado de Inteligencia Artificial (IA) generativa como apoyo a la codificación y consulta de recomendaciones, declaro haber utilizado la herramienta Gemini (modelo de lenguaje desarrollado por Google) exclusivamente para optimizar la lógica de sincronización offline masiva y para consultar recomendaciones de seguridad al procesar archivos JSON en Django REST Framework.
A continuación, se detallan los prompts exactos utilizados para este fin y cómo fueron aplicados en el proyecto.
Prompts exactos utilizados

Prompt 1 (Optimización del Desafío Offline inserción masiva):

"Necesito optimizar la lógica de un endpoint en Django REST Framework para un desafío offline de sincronización masiva. El endpoint recibe un JSON con múltiples registros de libros para insertar en la base de datos. ¿Qué código me recomiendas para hacer esto eficiente y asegurar que no se inserten datos corruptos?"

Respuesta y aplicación:

La IA recomendó utilizar `transaction.atomic()` de Django, el cual asegura la atomicidad de la base de datos al momento de guardar. Esto significa que si al procesar la lista de objetos masivos alguno falla (por ejemplo, porque los datos están corruptos o faltan campos obligatorios), la transacción se revierte de forma completa ("rollback"). Así, evitamos que la base de datos quede en un estado inconsistente con inserciones a medias.

Implementado en: catalogoApp/views.py -> sincronizar_libros

Prompt 2 (Recomendaciones de seguridad):

"Y algunas recomendaciones de seguridad al procesar estos archivos JSON masivos en Django."

Respuesta y aplicación:

1. **Validación estricta de esquemas y tipos de datos:** Evitar la inyección de datos maliciosos o corruptos validando todos los campos antes de procesarlos.

Implementado en: Se utilizó serializer.is_valid() asegurando que DRF filtre los campos contra el LibroSerializer antes de cualquier guardado).

2. **Uso de Transacciones Seguras (Atomicidad):** Prevenir la fragmentación e inconsistencia si ocurre un error a la mitad del procesamiento.
Implementado en: Se implementó transaction.atomic() en la vista de sincronización).

Capítulo 2
Referencias bibliográficas

La IA utilizada para resolver las problemáticas declaradas en los prompts anteriores corresponde a la siguiente fuente:

Google. (2026). *Gemini* (3.1 Pro) [Modelo de lenguaje grande]. https://gemini.google.com